현대 BFT 합의 알고리즘: 이론과 실전

작성일: 2025-01-07 **분석 대상**: CometBFT, HotStuff, Narwhal-Bullshark, DAG-based Consensus **목적**: 현대 BFT 합의 알고리즘의 이론적 기초와 구현 분석

목차

1. BFT 합의 기초
2. CometBFT 상세 분석
3. HotStuff 프로토콜
4. Narwhal-Bullshark
5. DAG 기반 합의
6. 성능 비교
7. Trade-offs 분석

---

1. BFT 합의 기초

1.1 Byzantine Fault Tolerance (BFT)

Byzantine Generals Problem:

- 1982년 Lamport, Shostak, Pease가 제안
- 악의적(Byzantine) 노드가 존재해도 정직한 노드들이 합의 도달

핵심 정리:

```
n >= 3f + 1

n: 총 노드 수
f: 악의적 노드 최대 수
```

Safety vs Liveness:

- **Safety**: 모든 정직한 노드가 같은 값에 합의
- **Liveness**: 합의가 반드시 종료됨

1.2 전통적 PBFT

Practical Byzantine Fault Tolerance (Castro & Liskov, 1999):

```
[Client] ──Request──> [Primary/Leader]
                          │
          ┌───────────────┼───────────────┐
          ▼               ▼               ▼
    ┌──────────┐    ┌──────────┐    ┌──────────┐
    │ Replica 1│    │ Replica 2│    │ Replica 3│
    └──────────┘    └──────────┘    └──────────┘

단계:
1. Pre-prepare: Leader가 요청 전파
2. Prepare: 2f+1 노드가 Prepare 메시지
3. Commit: 2f+1 노드가 Commit 메시지
4. Reply: 클라이언트에 응답
```

메시지 복잡도: $O(n^2)$

- 모든 노드가 모든 노드에게 메시지 전송
- 대규모 네트워크에서 병목

1.3 BFT 합의의 발전

```
1999: PBFT (O(n²))
  │
  ▼
2007: Zyzzyva (O(n) optimistic)
  │
  ▼
2014: Tendermint (O(n²), blockchain optimized)
  │
  ▼
2018: HotStuff (O(n), linear)
  │
  ▼
2021: Narwhal-Bullshark (DAG-based)
  │
  ▼
2024: Autobahn (Pipelined DAG)
```

---

2. CometBFT 상세 분석

2.1 아키텍처 개요

CometBFT (구 Tendermint)는 **Tendermint BFT** 합의 알고리즘을 구현합니다.

```go
// cometbft/internal/consensus/state.go (lines 72-144)
// 실제 State 구조체

type State struct {
    service.BaseService

    // config details
    config        *cfg.ConsensusConfig
    privValidator types.PrivValidator // for signing votes

    // store blocks and commits
    blockStore sm.BlockStore

    // create and execute blocks
    blockExec *sm.BlockExecutor

    // notify us if txs are available
    txNotifier txNotifier

    // add evidence to the pool
    evpool evidencePool

    // internal state
    mtx cmtsync.RWMutex
    cstypes.RoundState
    state sm.State // State until height-1.
    // privValidator pubkey, memoized for the duration of one block
    privValidatorPubKey crypto.PubKey

    // state changes may be triggered by: msgs from peers,
    // msgs from ourself, or by timeouts
    peerMsgQueue     chan msgInfo
    internalMsgQueue chan msgInfo
    timeoutTicker    TimeoutTicker

    // we use eventBus to trigger msg broadcasts in the reactor
    eventBus *types.EventBus

    // a Write-Ahead Log ensures we can recover from any kind of crash
    wal          WAL
    replayMode   bool
    doWALCatchup bool

    // for reporting metrics
    metrics *Metrics
}
```

2.2 합의 단계 (Round)

CometBFT의 각 라운드는 4단계로 구성됩니다:

```
┌──────────────────────────────────────────────────────────┐
│                        Round N                           │
├──────────┬──────────┬──────────────┬────────────────────┤
│ Propose  │ Prevote  │  Precommit   │      Commit        │
│ (제안)   │ (투표1)  │   (투표2)    │     (확정)         │
└──────────┴──────────┴──────────────┴────────────────────┘
     │          │           │              │
     │          │           │              │
     ▼          ▼           ▼              ▼
  Proposer   2/3+ votes   2/3+ votes     Block 최종화
  블록 제안   prevotes    precommits
```

enterNewRound

```go
// cometbft/internal/consensus/state.go (lines 1063-1127)
// 실제 enterNewRound 구현

func (cs *State) enterNewRound(height int64, round int32) {
    logger := cs.Logger.With("height", height, "round", round)

    if cs.Height != height || round < cs.Round ||
       (cs.Round == round && cs.Step != cstypes.RoundStepNewHeight) {
        logger.Debug("Entering new round with invalid args",
            "current", log.NewLazySprintf("%v/%v/%v", cs.Height, cs.Round, cs.Step))
        return
    }

    prevHeight, prevRound, prevStep := cs.Height, cs.Round, cs.Step

    // increment validators if necessary
    validators := cs.Validators
    if cs.Round < round {
        validators = validators.Copy()
        // Proposer 우선순위 증가 (라운드 변경 시)
        validators.IncrementProposerPriority(cmtmath.SafeSubInt32(round, cs.Round))
    }

    // Setup new round
    cs.updateRoundStep(round, cstypes.RoundStepNewRound)
    cs.Validators = validators

    // Round 0이 아니면 Proposal 리셋
    if round != 0 {
        logger.Info("Resetting proposal info", "proposer", propAddress)
        cs.Proposal = nil
        cs.ProposalReceiveTime = time.Time{}
        cs.ProposalBlock = nil
        cs.ProposalBlockParts = nil
    }

    // 다음 라운드 투표도 추적 (라운드 스킵 허용)
    cs.Votes.SetRound(cmtmath.SafeAddInt32(round, 1))
    cs.TriggeredTimeoutPrecommit = false

    // Wait for txs or enterPropose immediately
    waitForTxs := cs.config.WaitForTxs() && round == 0 && !cs.needProofBlock(height)
    if waitForTxs {
        if cs.config.CreateEmptyBlocksInterval > 0 {
            cs.scheduleTimeout(cs.config.CreateEmptyBlocksInterval, height, round,
                cstypes.RoundStepNewRound)
        }
        return
    }

    cs.enterPropose(height, round)
}
```

enterPropose

```go
// cometbft/internal/consensus/state.go (lines 1152-1212)
// 실제 enterPropose 구현

func (cs *State) enterPropose(height int64, round int32) {
    logger := cs.Logger.With("height", height, "round", round)

    if cs.Height != height || round < cs.Round ||
       (cs.Round == round && cstypes.RoundStepPropose <= cs.Step) {
        return
    }

    // PBTS (Proposer-Based TimeStamp) 처리
    if cs.isPBTSEnabled(height) && cs.privValidatorPubKey != nil &&
       cs.isProposer(cs.privValidatorPubKey.Address()) {
        proposerWaitTime := proposerWaitTime(cmttime.DefaultSource{}, cs.state.LastBlockTime)
        if proposerWaitTime > 0 {
            cs.scheduleTimeout(proposerWaitTime, height, round, cstypes.RoundStepNewRound)
            return
        }
    }

    defer func() {
        // Done enterPropose
        cs.updateRoundStep(round, cstypes.RoundStepPropose)
        cs.newStep()

        // Proposal이 완전하면 바로 Prevote로
        if cs.isProposalComplete() {
            cs.enterPrevote(height, cs.Round)
        }
    }()

    // Propose 타임아웃 설정
    cs.scheduleTimeout(cs.config.Propose(round), height, round, cstypes.RoundStepPropose)

    // Validator가 아니면 제안 안 함
    if cs.privValidator == nil {
        logger.Debug("Propose step; not proposing since node is not a validator")
        return
    }

    addr := cs.privValidatorPubKey.Address()

    // Validator set에 없으면 제안 안 함
    if !cs.Validators.HasAddress(addr) {
        logger.Debug("Propose step; not proposing since node is not in the validator set")
        return
    }
}
```

enterPrevote

```go
// cometbft/internal/consensus/state.go (lines 1345-1369)
// 실제 enterPrevote 구현

func (cs *State) enterPrevote(height int64, round int32) {
    logger := cs.Logger.With("height", height, "round", round)

    if cs.Height != height || round < cs.Round ||
       (cs.Round == round && cstypes.RoundStepPrevote <= cs.Step) {
        logger.Debug("Entering prevote step with invalid args",
            "current", log.NewLazySprintf("%v/%v/%v", cs.Height, cs.Round, cs.Step))
        return
    }

    defer func() {
        // Done enterPrevote
        cs.updateRoundStep(round, cstypes.RoundStepPrevote)
        cs.newStep()
    }()

    logger.Debug("Entering prevote step",
        "current", log.NewLazySprintf("%v/%v/%v", cs.Height, cs.Round, cs.Step))

    // Sign and broadcast vote
    cs.doPrevote(height, round)

    // 2/3+ prevotes 도달하면 PrevoteWait으로
}
```

defaultDoPrevote

```go
// cometbft/internal/consensus/state.go (lines 1387-1494)
// 실제 Prevote 로직 (의사코드와 대응)

func (cs *State) defaultDoPrevote(height int64, round int32) {
    logger := cs.Logger.With("height", height, "round", round)

    // Proposal이 없으면 nil 투표
    if cs.Proposal == nil {
        logger.Debug("Prevote step: did not receive a valid Proposal; prevoting nil")
        cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
        return
    }

    // ProposalBlock이 없으면 nil 투표
    if cs.ProposalBlock == nil {
        logger.Debug("Prevote step: did not receive the ProposalBlock; prevoting nil")
        cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
        return
    }

    /*
        의사코드 (Tendermint 논문):
        22: upon <PROPOSAL, h_p, round_p, v, -1> from proposer(h_p, round_p)
            while step_p = propose do
        23: if valid(v) && (lockedRound_p = -1 || lockedValue_p = v) then
        24:   broadcast <PREVOTE, h_p, round_p, id(v)>
        25: else
        26:   broadcast <PREVOTE, h_p, round_p, nil>
    */
    if cs.Proposal.POLRound == -1 {
        if cs.LockedRound == -1 {
            // 유효한 블록이 있고 일치하면 투표
            if cs.ValidRound != -1 && cs.ProposalBlock.HashesTo(cs.ValidBlock.Hash()) {
                logger.Debug("Prevote step: ProposalBlock matches our valid block")
                cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(),
                    cs.ProposalBlockParts.Header(), nil)
                return
            }

            // PBTS (Proposer-Based TimeStamp) 검증
            if cs.isPBTSEnabled(height) {
                if !cs.Proposal.Timestamp.Equal(cs.ProposalBlock.Header.Time) {
                    logger.Debug("Prevote step: proposal timestamp not equal; prevoting nil")
                    cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
                    return
                }

                if !cs.proposalIsTimely() {
                    logger.Info("Prevote step: Proposal is not timely; prevoting nil")
                    cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
                    return
                }
            }

            // 블록 검증 (합의 관점)
            err := cs.blockExec.ValidateBlock(cs.state, cs.ProposalBlock)
            if err != nil {
                logger.Error("prevote step: consensus deems this block invalid; prevoting nil")
                cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
                return
            }

            // ProcessProposal ABCI 호출 (애플리케이션 검증)
            isAppValid, err := cs.blockExec.ProcessProposal(cs.ProposalBlock, cs.state)
            if !isAppValid {
                logger.Error("Prevote step: state machine rejected proposed block")
                cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{}, nil)
                return
            }

            // 모든 검증 통과 - 블록에 투표
            cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(),
                cs.ProposalBlockParts.Header(), nil)
        }
        // ... LockedRound 처리
    }
```

```
    // ... POLRound >= 0 처리
}
```

2.3 Locking 메커니즘

CometBFT는 **Locking**을 통해 Safety를 보장합니다:

```
┌──────────────────────────────────────────────────────────┐
│                      Locking Rules                       │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  1. LockedValue 설정:                                    │
│     - 2/3+ precommits를 받으면 해당 블록에 lock          │
│                                                          │
│  2. LockedValue 해제:                                    │
│     - 더 높은 라운드에서 2/3+ prevotes (POL) 받으면 unlock │
│                                                          │
│  3. 투표 규칙:                                           │
│     - Lock된 상태: 반드시 LockedValue에 투표             │
│     - Lock 안 된 상태: 유효한 Proposal에 투표            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```go
// Locking 상태 변수
type RoundState struct {
    LockedRound int32        // Lock된 라운드 (-1이면 없음)
    LockedBlock *types.Block // Lock된 블록
    ValidRound  int32        // Valid 라운드 (-1이면 없음)
    ValidBlock  *types.Block // Valid 블록

    // POL (Proof Of Lock)
    // 2/3+ prevotes의 증거
}
```

2.4 메시지 복잡도

```
CometBFT 메시지 복잡도:

Propose:    O(n)     - Proposer → All
Prevote:    O(n²)    - All → All
Precommit:  O(n²)    - All → All
──────────────────────
Total:      O(n²)

n = validator 수
```

실제 메시지 수 (n=100):

- Propose: 100 메시지
- Prevote: 100 × 100 = 10,000 메시지
- Precommit: 100 × 100 = 10,000 메시지
- **총**: ~20,000 메시지/블록

---

3. HotStuff 프로토콜

3.1 핵심 혁신

HotStuff (Yin et al., 2018)는 **선형 메시지 복잡도** O(n)를 달성합니다.

핵심 아이디어:

1. **Threshold Signatures**: 2/3+ 투표를 단일 서명으로 집계
2. **Pipelining**: 여러 블록을 병렬 처리
3. **Three-phase Commit**: Prepare → Pre-commit → Commit

3.2 프로토콜 구조

```
┌─────────────────────────────────────────────────────────┐
│                    HotStuff 3-Phase                     │
├─────────────────────────────────────────────────────────┤
│                                                         │
│   Phase 1: PREPARE                                      │
│   Leader ──proposal──> Replicas                         │
│   Replicas ──vote──> Leader                             │
│   Leader: prepareQC = aggregate(votes)                  │
│                                                         │
│   Phase 2: PRE-COMMIT                                   │
│   Leader ──prepareQC──> Replicas                        │
│   Replicas ──vote──> Leader                             │
│   Leader: precommitQC = aggregate(votes)                │
│                                                         │
│   Phase 3: COMMIT                                       │
│   Leader ──precommitQC──> Replicas                      │
│   Replicas ──vote──> Leader                             │
│   Leader: commitQC = aggregate(votes)                   │
│                                                         │
│   Phase 4: DECIDE                                       │
│   Leader ──commitQC──> Replicas                         │
│   Replicas: commit block                                │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

3.3 Chained HotStuff

Pipelining 최적화:

```
View 1:  [B1] ─prepare──────────────────────────────────────────
View 2:       [B2] ─prepare──────> [B1] ─pre-commit──────────────
View 3:            [B3] ─prepare──────> [B2] ─pre-commit──────> [B1] ─commit
View 4:                 [B4] ─prepare──────> [B3] ─pre-commit──────> [B2] ─commit
                                                                     [B1] decide!

한 라운드가 여러 블록의 서로 다른 단계를 처리
```

3.4 의사코드

```
// HotStuff Basic Protocol

function onReceiveProposal(proposal, prepareQC):
    if proposal.view ≠ currentView:
        return

    if not safeNode(proposal, prepareQC):
        return

    vote = sign(proposal)
    send(vote, leader)

function safeNode(proposal, prepareQC):
    // Safety rule: 이전 QC가 있거나 prepareQC가 lockedQC보다 높아야 함
    return (prepareQC.view > lockedQC.view) or
           (proposal extends lockedQC.block)

function onReceiveVotes(votes):
    if |votes| >= 2f + 1:
        qc = aggregate(votes)  // Threshold signature
        switch qc.type:
            case PREPARE:
                prepareQC = qc
                broadcast(prepareQC)
            case PRE-COMMIT:
                precommitQC = qc
                lockedQC = qc  // Lock on this block
                broadcast(precommitQC)
            case COMMIT:
                commitQC = qc
                decide(qc.block)
```

3.5 vs CometBFT 비교

특성	CometBFT	HotStuff
메시지 복잡도	$O(n^2)$	$O(n)$
위상 수	3 (Prevote, Precommit, Commit)	3 (Prepare, Pre-commit, Commit)
Leader 의존	중간	높음
View Change	타임아웃 기반	QC 기반
Pipelining	제한적	네이티브 지원
구현 복잡도	낮음	중간

---

4. Narwhal-Bullshark

4.1 핵심 아이디어

Narwhal-Bullshark는 **합의와 데이터 전파를 분리**합니다.

```
┌──────────────────────────────────────────────────────────┐
│              Narwhal + Bullshark Architecture            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────────────────────────────┐       │
│   │                   Bullshark                  │       │
│   │               (Consensus Layer)              │       │
│   │                                              │       │
│   │   • DAG 순서화                               │       │
│   │   • Leader 선출                              │       │
│   │   • 합의 도달                                │       │
│   └──────────────────────────────────────────────┘       │
│                          ▲                               │
│                          │ DAG vertices                  │
│                          │                               │
│   ┌──────────────────────────────────────────────┐       │
│   │                   Narwhal                    │       │
│   │              (Data Availability)             │       │
│   │                                              │       │
│   │   • Transaction 수집                         │       │
│   │   • DAG 구축                                 │       │
│   │   • Data 전파                                │       │
│   └──────────────────────────────────────────────┘       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

4.2 Narwhal: DAG-based Mempool

```
                Round 4
                   │
     ┌─────────────┼─────────────┐
     │             │             │
     ▼             ▼             ▼
┌─────────┐   ┌─────────┐   ┌─────────┐
│ V4,1    │   │ V4,2    │   │ V4,3    │
└─────────┘   └─────────┘   └─────────┘
     │    \     │     /     │
     │     \    │    /      │
     │      \   │   /       │
     ▼       ▼  ▼  ▼        ▼
┌─────────┐   ┌─────────┐   ┌─────────┐
│ V3,1    │───│ V3,2    │───│ V3,3    │   Round 3
└─────────┘   └─────────┘   └─────────┘

V = Vertex (트랜잭션 배치 + 이전 라운드 참조)
각 Validator는 라운드당 하나의 vertex 생성
```

4.3 Bullshark: Zero-message Overhead Consensus

```
// Bullshark 합의 알고리즘

function processVertex(v):
    if v.round == 0:
        return

    // 앵커 결정 (라운드 리더의 vertex)
    if isLeader(v.author, v.round):
        if hasStrongSupport(v):  // 2f+1 노드가 참조
            anchor = v
            commitAnchor(anchor)

function commitAnchor(anchor):
    // 앵커의 causal history 커밋
    for vertex in causalHistory(anchor):
        if not committed(vertex):
            commit(vertex)

function hasStrongSupport(v):
    nextRound = v.round + 1
    supporters = 0
    for vertex in vertices[nextRound]:
        if v in vertex.parents:
            supporters++
    return supporters >= 2f + 1
```

4.4 성능 특성

메트릭	값
TPS	160,000+ (optimal conditions)
메시지 복잡도	O(n) - consensus, $O(n^2)$ - data
지연 시간	3 라운드
장애 허용	f < n/3

---

5. DAG 기반 합의

5.1 DAG (Directed Acyclic Graph) 구조

```
                    Time →

Round 4:  [A₄] ──────────────────────────
            | \
            |  \
Round 3:  [A₃]──[B₃]──[C₃] ──────────────
            | /   | \    \
            |/    |  \    \
Round 2:  [A₂]──[B₂]──[C₂] ──────────────
            | /   | /
            |/    |/
Round 1:  [A₁]──[B₁]──[C₁] ──────────────

각 블록은 이전 라운드의 2f+1 블록을 참조
```

5.2 DAG 합의의 장점

1. **병렬 처리**: 여러 validator가 동시에 블록 생성
2. **높은 처리량**: 순차 병목 제거
3. **공정성**: 모든 트랜잭션이 최종적으로 포함
4. **Leader 없는 구조**: 단일 실패점 없음

5.3 DAG 순서화 규칙

```
// DAG 순서화 (Topological Sort with Determinism)

function orderDAG(dag):
    ordered = []

    // 1. 각 라운드별로 처리
    for round in rounds(dag):
        // 2. 앵커 (리더 블록) 확인
        anchor = findAnchor(round)

        if anchor != nil and hasQuorum(anchor):
            // 3. Causal history 정렬
            history = causalHistory(anchor)
            sortedHistory = topologicalSort(history)

            // 4. 결정적 순서로 추가
            for block in sortedHistory:
                if block not in ordered:
                    ordered.append(block)

    return ordered

function topologicalSort(blocks):
    // 동일 라운드 내: 해시 순서로 정렬 (결정적)
    // 다른 라운드 간: 라운드 순서 유지
    return stableSort(blocks, by: (round, hash))
```

5.4 DAG vs Chain

특성	Chain (CometBFT)	DAG (Narwhal)
블록 생성	순차 (한 번에 하나)	병렬 (라운드당 n개)
처리량	리더 병목	대역폭 한계
지연 시간	일정	가변 (최적화 가능)
공정성	리더 의존	내재적 공정성
복잡도	낮음	높음

---

6. 성능 비교

6.1 이론적 복잡도

알고리즘	메시지 복잡도	라운드 복잡도	View Change
PBFT	$O(n^2)$	O(1)	$O(n^2)$
CometBFT	$O(n^2)$	O(1)	O(n)
HotStuff	O(n)	O(1)	O(n)
Bullshark	O(n)	O(1)	N/A (leaderless)

6.2 실제 성능 (벤치마크)

알고리즘	TPS (4 노드)	TPS (100 노드)	지연 시간
CometBFT	~10,000	~1,000	~1초
HotStuff	~50,000	~10,000	~0.5초
Bullshark	~100,000	~50,000	~0.3초

주의: 실제 성능은 네트워크, 하드웨어, 워크로드에 따라 다름

6.3 Validator 수에 따른 확장성

```
TPS
 │
 │  Bullshark
 │  ┌──────────────────────────────
 │ /
 │/     HotStuff
 │  ┌──────────────────────
 │ /
 │/
 │      CometBFT
 │  ┌──────────────
 │ /    \
 │/      \────
 │
 └──────────────────────────────── Validators
        10      50      100     200
```

7. Trade-offs 분석

7.1 안전성 vs 활성

```
┌─────────────────────────────────────────────────────────┐
│                   Safety vs Liveness                    │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  Safety 우선 (CometBFT):                                │
│  - 네트워크 파티션 시 멈춤                              │
│  - 결코 잘못된 값에 합의하지 않음                       │
│  - 2/3+ 가용성 필요                                     │
│                                                         │
│  Liveness 우선:                                         │
│  - 항상 진행 (eventually)                               │
│  - 비동기 네트워크에서도 동작                           │
│  - 더 복잡한 프로토콜                                   │
│                                                         │
│  균형점:                                                │
│  - 부분 동기 가정 (Partial Synchrony)                   │
│  - GST (Global Stabilization Time) 이후 진행            │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

7.2 처리량 vs 분산화

```
┌─────────────────────────────────────────────────────────┐
│             Throughput vs Decentralization              │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  높은 처리량 (예: Solana):                              │
│  - 적은 validator (~1000)                               │
│  - 높은 하드웨어 요구사항                               │
│  - 중앙화 경향                                          │
│                                                         │
│  높은 분산화 (예: Bitcoin):                             │
│  - 많은 노드 (~10000+)                                  │
│  - 낮은 처리량                                          │
│  - 검열 저항성                                          │
│                                                         │
│  균형점:                                                │
│  - 100-500 validators                                   │
│  - 1000-10000 TPS                                       │
│  - BFT 합의                                             │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

7.3 복잡도 vs 성능

측면	단순함 (CometBFT)	복잡함 (Narwhal)
구현	이해하기 쉬움	전문 지식 필요
디버깅	직관적	까다로움
감사	쉬움	어려움
성능	중간	높음
유지보수	낮은 비용	높은 비용

7.4 사용 사례별 권장

```
┌─────────────────────────────────────────────────────────────┐
│                    Use Case Recommendations                  │
├─────────────────────────────────────────────────────────────┤
│                                                              │
│  DeFi / Financial Applications:                              │
│  → CometBFT or HotStuff                                      │
│  • Safety 최우선                                             │
│  • 검증된 구현                                               │
│  • 적당한 처리량                                             │
│                                                              │
│  High-Throughput Applications:                               │
│  → Narwhal-Bullshark                                         │
│  • 게임, NFT 마켓플레이스                                    │
│  • 높은 TPS 요구                                             │
│  • 복잡도 감수 가능                                          │
│                                                              │
│  General-Purpose Blockchain:                                 │
│  → CometBFT (기본) or HotStuff (확장)                        │
│  • 균형잡힌 선택                                             │
│  • 생태계 도구 지원                                          │
│  • 점진적 최적화                                             │
│                                                              │
└─────────────────────────────────────────────────────────────┘
```

요약

핵심 포인트

1. **CometBFT**:
   - $O(n^2)$ 메시지 복잡도
   - 검증된, 안정적인 구현
   - Cosmos 생태계 표준
2. **HotStuff**:
   - O(n) 메시지 복잡도 (선형)
   - Threshold signatures로 집계
   - Pipelining으로 처리량 향상
3. **Narwhal-Bullshark**:
   - 합의와 데이터 분리
   - DAG 기반 병렬 처리
   - 최고 처리량
4. **Trade-offs**:
   - 복잡도 ↔ 성능
   - 안전성 ↔ 활성
   - 분산화 ↔ 처리량

다음 단계

1. **Phase 2 문서 06**: Autobahn BFT & Malachite 분석
2. **Phase 2 문서 07**: CometBFT 개선 제안서

작성 완료: 2025-01-07 **총 페이지**: ~18페이지 **분석 기반**: CometBFT 실제 소스코드, HotStuff/Narwhal 논문